Software Requirements Specification (SRS)

Project: Vaultory — Small Business Inventory and Sales App (SBISA)

Document ID	SRS-VAULTORY-001
Version	1.1
Status	Draft for Review & Sign-off
Prepared By	Anoop (Solutions Architect) — Vaultory
Date	29/08/2026
Client / Sponsor	Small Business Retailer (Prof)
Base Document	BRD v3.4 (BRD-VAULTORY-001)

Revision History

Version	Date	Author	Description of Change
1.0	29/08/2026	Anoop (Solutions Architect)	Initial SRS derived from BRD v3.3
1.1	29/08/2026	Anoop (Solutions Architect)	Locked stack per BRD v3.4: React+Tailwind+shadcn/ui, Node.js backend + AI via Groq API, Supabase (Postgres/Auth incl. email OTP/Storage); resolved TBD-1 (Node) & TBD-2 (shadcn + Recharts)

Approvals

Role / Designation	Name	Signature	Date
Client / Sponsor	Prof		
Project Manager	Laxman Patel		
Business Analyst	Ved Naik		
Solutions Architect	Anoop Gupta		
Scrum Master	Devdarshan S		
Tech Lead	Rohan Vashisht		

> **SIGN-OFF NOTICE:** This SRS is the **technical baseline** for building Vaultory. It translates every BRD requirement into concrete, testable functional specifications. It is binding. **Any behavior not described here is Out of Scope**, exactly as in the BRD, and requires a Change Request.

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes in technical detail **what the Vaultory application must do** so the development team (Tech Lead + team) can design, code, and test it, and the client can confirm the behavior. It is derived from, and fully consistent with, the **BRD v3.4**. Every requirement in the BRD is traced to a technical specification here (see Section 14).

1.2 Product Scope Summary

Vaultory is a web-based inventory and sales management application for a small retailer with **3 stores**. It provides:

- Multi-location inventory tracking (stores + warehouses).
- Sales recording with automatic stock deduction.
- Daily / quarterly / yearly sales reporting.
- Configurable safety stock & reorder points with alerts.
- **Automated AI ordering** when stock reaches the reorder point.
- **AI warehouse stock-level recommendations**.
- Store-wise sales performance views (sales personnel).
- Executive monitoring dashboards (senior stakeholders).
- Role-based access control (RBAC), data masking, audit logging.

1.3 Intended Audience

- **Client (Prof)** — to validate expected behavior.
- **Development team** — to implement and test.
- **QA / UAT** — to verify acceptance criteria.
- **PM / SM** — to estimate and plan (feeds the Sprint Planner).

---

2. Overall Description & Context

2.1 Product Perspective

Vaultory is an independent, self-contained web application. It does **not** integrate with external systems in this version (BRD §9). It consists of:

```
[ React SPA (Vercel) ]  <->  [ REST API + AI service (Render: Node.js) ]  <->  [ Supabase ]
                                                                              (PostgreSQL + Auth + Storage)
                                                                                     |
                                                                                     v
                                                                          [ Groq API (LLM reasoning / forecasting) ]
```

2.2 User Classes & Characteristics

Class	Description	Skill Level
Admin	Configures products, users, suppliers, safety stock, rules	Basic–Intermediate
Store Staff	Records stock-in/out/transfer/adjustments, receives goods	Basic
Sales Personnel	Records sales, views store performance	Basic
Senior Stakeholder	Views dashboards & reports	Basic

2.3 Operating Environment

- **Browser:** modern evergreen browsers (Chrome 90+, Firefox 90+, Edge 90+, Safari 14+).
- **Devices:** responsive — desktop, tablet, mobile browser.
- **Network:** internet required; HTTPS only.
- **Hosting:** Vercel (frontend), Render (backend/AI), Supabase (Postgres/Auth/Storage), Groq API (AI inference) — free tier (BRD §8.5). GCP/AWS are Phase-2 post-handover only.

2.4 Design & Implementation Constraints

1. Frontend: **React (TypeScript)** + **Tailwind CSS** + **shadcn/ui**; charts via **Recharts**.
2. Backend: **Node.js** (Express **or** NestJS). *(Locked at BRD v3.4 — no Python.)*
3. Database/backend services: **Supabase** for **PostgreSQL** + **Auth** (email/password + email OTP) + **Object Storage**.
4. AI: Node.js AI service calling the **Groq API** (LLM reasoning/forecasting).
5. Deployment: **Vercel + Render + Supabase** free tier (BRD §8.5); Groq API key as a secret env var on Render.
6. All sensitive product information **masked**.
7. Single currency, single language.

2.5 Assumptions & Dependencies

Same as BRD §17 (3 stores, single currency/language, manual sales entry, client-provided master data, Vercel/Render/Supabase free-tier accounts, Groq free-tier API access).

3. Scope & References

3.1 In Scope

Identical to BRD §8 — all modules listed in the BRD (Inventory, Sales, Safety Stock, Procurement, AI Ordering, AI Warehouse Recommendations, Monitoring, RBAC, value-add modules).

3.2 Out of Scope

Identical to BRD §9 — L-1 through L-21 (no mobile apps, ecommerce, payments, POS, CRM, accounting, supplier portal, multi-currency, etc.).

3.3 References

- **BRD v3.4** — Business Requirements Document (BRD-VAULTORY-001).
- Problem Statement — Product 5: Small Business Inventory and Sales App (22/08/2026).

4. Functional Requirements — Detailed Specifications

> This section expands each BRD functional requirement into **concrete technical behavior**: inputs, processing rules, outputs, validation, and error handling. Requirement IDs align with the BRD for traceability (`FR-<MODULE>-<seq>` = BRD ID; `.n` = technical sub-spec).

4.1 MODULE: Inventory Management

4.1.1 FR-INV-01 — Product (SKU) Master

- **Fields:** `sku_code` (unique, required, max 32 chars, auto-suggest pattern `[P]<category>-<seq>`), `name` (required, max 120), `description` (optional, max 500), `category_id` (FK), `unit_id` (FK), `cost_price` (masked, decimal(12,2)), `sale_price` (decimal(12,2)), `default_safety_stock` , `default_reorder_point` , `default_target_level` (integers ≥0), `status` (active/archived), timestamps.
- **Validation rules:**
  - `sku_code` unique — reject duplicates with error.
  - `sale_price ≥ 0` , `cost_price ≥ 0` , `target ≥ reorder ≥ safety` .
  - Soft-deactivate (archive) only; never hard delete. Archived SKU blocked from new transactions; retained in history/reports.
- **Errors:** duplicate SKU → `409 SKU_ALREADY_EXISTS` ; invalid price → `422 INVALID_PRICE` .
- **Permissions:** create/edit → Admin only (BRD §12).

4.1.2 FR-INV-02 — Location Master (Stores & Warehouses)

- `locations` type ENUM: `store | warehouse` .
- Seed data: 3 stores (`Store A` , `Store B` , `Store C`) + 1 warehouse (`Central Warehouse`).
- Fields: `name` , `city` , `address` , `status` (active/inactive).
- Adding extra stores beyond 3 = Change Request (BRD L-14).

4.1.3 FR-INV-03 — Stock-on-Hand Tracking

- Table `inventory` : composite unique `(product_id, location_id)` , `qty_on_hand` (integer, ≥0).
- Every transaction (stock-in/out/transfer/adjustment/sale) writes a `stock_movements` audit row and updates `qty_on_hand` accordingly.
- Stock view (read-only for authorized roles) returns: on-hand, reorder_point, safety_stock, target_level, status badge.
- **Status logic:**
  - `OUT` : qty == 0
  - `LOW` : 0 < qty ≤ reorder_point
  - `IN` : reorder_point < qty ≤ target_level
  - `OVER` : qty > target_level

4.1.4 FR-INV-04 — Stock-In

- Input: `product_id` , `qty > 0` , `destination_location_id` , optional `po_id` , optional notes.
- Effect: `qty_on_hand += qty` at destination; if `po_id` , update PO received status (FR-PRO-05).
- Guard: destination active; qty integer > 0.

4.1.5 FR-INV-05 — Stock-Out

- Input: `product_id` , `qty > 0` , `source_location_id` , `reason` (damage/loss/other), notes.
- Effect: `qty_on_hand -= qty` .
- Guard: **no negative stock** — if qty > on-hand, reject (`409 INSUFFICIENT_STOCK`) by default. (Config can downgrade to warn; default = block.)

4.1.6 FR-INV-06 — Stock Transfer

- Input: `product_id` , `qty` , `from_location_id` , `to_location_id` (must differ).
- Effect: `from -= qty` , `to += qty` in a single transaction (ACID); net zero.

- Guard: qty ≤ from-location on-hand.

4.1.7 FR-INV-07 — Stock Adjustment (Cycle Count)

- Input: `product_id`, `location_id`, `counted_qty` (≥0), `reason` (required).
- Behavior: compute `variance = counted_qty − qty_on_hand`. On confirm, set `qty_on_hand = counted_qty`, log adjustment with reason, actor, timestamp.
- Variance shown **before** confirmation.

4.1.8 FR-INV-08 — Status & Alerts

- After every stock mutation, re-evaluate status.
- On transition to `LOW` or `OUT`: create `alerts` row (`type` = `LOW_STOCK` / `OUT_OF_STOCK`, target roles = Admin + relevant store staff).
- Alerts marked `read` / `unread`; list view in Alerts Center.

4.2 MODULE: Sales Management

4.2.1 FR-SAL-01 — Sale Recording

- Input: `store_id`, `sale_datetime` (server time default), line items: `[{product_id, qty>0, unit_price}]`.
- Rules:
  - ≥1 line item required.
  - `line_total = qty × unit_price`; `sale.total = Σ line_total` (server computed; client-sent totals ignored).
  - **Stock deduction:** for each line, `inventory.qty_on_hand −= qty` at the sale's store.
  - If qty > on-hand → **block** by default (`409 INSUFFICIENT_STOCK`); identifies which product.
  - Sale is immutable after creation except void (FR-SAL-04).
- Audit: each sale + stock movement recorded.

4.2.2 FR-SAL-02 — Sales Reports (Day / Quarter / Year)

- Endpoints/reports:
  - **Daily:** filter `store_id, product_id, date`.
  - **Quarterly:** filter `store_id, product_id, quarter (YYYY-Qq)`.
  - **Yearly:** filter `store_id, product_id, year (YYYY)`.
- Output metrics: `units_sold`, `sales_value`, grouped by store and product; sortable; export CSV/PDF.
- Auto-generation: daily summary snapshot stored; quarterly & yearly aggregate queries on demand.

4.2.3 FR-SAL-03 — Store-wise Performance

- Sales personnel view: per-store dashboard (value, units, period); side-by-side store comparison; top products per store.

4.2.4 FR-SAL-04 — Void / Returns

- **Void:** only authorized users (Admin); require `reason`. Effect: reverse stock (`+= qty`), exclude sale from totals, mark `status=voided`, audit log.
- **Return:** recorded as negative sale line: stock increases, sales totals reduce; reason captured.

4.3 MODULE: Safety Stock Management

4.3.1 FR-SST-01 — Configuration

- Per-product, per-location (or global default) values: `safety_stock`, `reorder_point`, `target_level`, `auto_order_enabled` (bool).
- Validation: `target_level ≥ reorder_point ≥ safety_stock ≥ 0`.
- Sources: manual entry **or** AI suggestion (accept → writes the suggested values).

4.3.2 FR-SST-02 — Tracking & Alerts

- Continuous comparison (after each mutation) of `qty_on_hand` vs `reorder_point`.
- `qty ≤ reorder_point` → product flagged + reorder evaluation triggered (Section 8.1) + alert created (target Admin; also relevant staff).

4.4 MODULE: Supplier & Procurement

4.4.1 FR-PRO-01 — Supplier Master

- Fields: `name`, `contact_person`, `phone`, `email`, `address`, `lead_time_days` (int >0), `status`, masked finance fields (e.g., credit terms).
- Mapping: `supplier_products(product_id, supplier_id)` many-to-many.

4.4.2 FR-PRO-02 / FR-PRO-03 — Purchase Order (Manual & Auto)

- PO fields: `po_number` (auto `PO-<YYYY>-<seq>`), `supplier_id`, `destination_id`, `status`, `order_date`, `expected_date = order_date + lead_time_days`, line items `{product_id, qty, received_qty}`.
- **Auto-PO** generated by AI flow (Section 8.1).
- **Manual PO** created by authorized user with validated qty > 0.

4.4.3 FR-PRO-04 — PO Lifecycle

- Status flow: `DRAFT → SENT → PARTIALLY_RECEIVED → RECEIVED → CLOSED` any state → `CANCELLED`.
- State transitions logged (actor, timestamp, old, new).

4.4.4 FR-PRO-05 — PO Receipt

- Goods-in against PO: `received_qty` per line; stock increase at destination.
- If all lines fully received → `RECEIVED`; partial → `PARTIALLY_RECEIVED`.
- Over-receipt guard: `received_qty + previous_received ≤ qty`.

4.4.5 FR-PRO-06 — Duplicate Prevention

- Auto-PO suppressed if an open (non-closed/cancelled) PO for same `(product_id, destination_id)` already exists, unless config override.

4.5 MODULE: Value-Add Modules (BRD §8.1 / FR-CAT…FR-ONB)

4.5.1 FR-CAT — Categories & Units

- `categories` tree (self-referencing parent_id), `units` list. Assign product to one category + one unit.
- Reports group by category/unit. In-use values cannot be hard-deleted (soft archive).

4.5.2 FR-SUP — Supplier Performance (Value-add)

- Compute on-time delivery % per supplier over rolling window: `on_time = received_date ≤ expected_date`.
- Effective lead time for AI = rolling average of actual lead times (fall back to nominal if insufficient data).

4.5.3 FR-DSH — Dashboard Widgets

- KPI cards: total stock value, inventory turnover, low-stock count, out-of-stock count, today's sales.
- Role-constrained widget set; click-through drill-down.

4.5.4 FR-FSM — Fast/Slow Movers

- Classification by sales velocity (units sold per period) over rolling window (e.g., 90 days).
- Thresholds configurable (defaults: Fast ≥ threshold A, Slow ≤ threshold B).
- Slow movers flagged; fast movers prioritized in AI reorder/warehouse.

4.5.5 FR-EXP — Perishable Flag (advisory only)

- `is_perishable` bool + `shelf_life_days` (int). Warning/flag when estimated remaining shelf-life low. **No** batch/lot tracking (guardrail per BRD §10.13).
- Not in v1.0 core: marked `C` (Could) — delivered only if sprint capacity allows (see Sprint Planner).

4.5.6 FR-BULK — Import/Export

- Export: CSV of products, inventory, sales, POs (respect caller's masking permission).
- Import: products & opening stock via CSV template; server-side validation; per-row results (added/updated/failed with reasons); import session in audit log.
- Limits: max 5,000 rows per import; file ≤ 10 MB.

4.5.7 FR-ALR — Alert Preferences

- Per-user in-app preference toggles: low-stock, PO, expiry.
- Optional email alert to Admin/Seniors (config). No SMS/customer messaging.

4.5.8 FR-AUD — Audit Log Viewer

- Append-only log: actor, action, entity, entity_id, detail (JSON), created_at, ip (optional).
- Admin view with filters (actor, action, date range, entity).
- Masked sensitive values in logs.

4.5.9 FR-ONB — Onboarding Wizard

- Guided sequence: Location setup → Users → Products → Suppliers & mapping → Opening stock → Safety stock.
- Skippable, resumable, per-step validation with clear errors.

4.6 MODULE: User Management & RBAC

4.6.1 FR-USER-01 — Authentication

- Login via **Supabase Auth**: **email/password** and **email OTP (magic link)** passwordless sign-in, both enabled.
- Email OTP: Supabase sends a one-time code / magic link to the user's email; code expires per Supabase config (default 3–10 min) and is single-use.
- Passwords and OTP handling are managed by **Supabase Auth** (not implemented in the Node backend).
- Session: Supabase-issued **JWT (access token)** validated by the **Node backend** role middleware on every protected route; refresh token handled by Supabase (default 1-hour access + refresh lifecycle).
- Logout invalidates the Supabase session.
- Rate-limit / lockout: configure in **Supabase Auth** settings (e.g., max failed attempts before cooldown).

4.6.2 FR-USER-02 — Roles & Permissions

- Roles ENUM: `ADMIN`, `STORE_STAFF`, `SALES_PERSONNEL`, `SENIOR_STAKEHOLDER`.
- Protect **server-side** every route/method per BRD §12 matrix. Client hides menus; server enforces.
- User has optional `store_id` (store staff scoped to their store).

4.6.3 FR-USER-03 — User Administration (Admin only)

- CRUD users (name, email, role, store, password reset), activate/deactivate.
- Cannot deactivate self or delete users (soft).

---

5. External Interface Requirements

5.1 User Interfaces

Web UI (responsive). Full per-screen specs in Section 11.

5.2 Hardware Interfaces

None (no POS, scanners, printers — BRD L-4).

5.3 Software Interfaces

- **REST API** between frontend and backend (JSON over HTTPS). Standards: RESTful resources, snake_case JSON, ISO-8601 timestamps, standardized error envelope `{ error: { code, message, details? } }`.
- **DB interface:** Node backend connects to **Supabase PostgreSQL** using the **Supabase JS/Postgres client** (or a Node driver/ORM such as Prisma) over the Supabase connection string.
- **AI module:** Node.js AI service that calls the **Groq API** (external LLM endpoints via HTTPS) for forecasting/recommendation reasoning, using secrets from env vars; deterministic rules stay in the Node backend.

5.4 Communication Interfaces

- HTTPS only (TLS 1.2+). CORS restricted to the Vercel frontend origin. Rate limiting on auth endpoints.

---

6. Data & Database Specifications

6.1 Schema (PostgreSQL via Supabase)

> **Auth note:** user authentication/sessions live in **Supabase Auth** (`auth.users`), which also issues the OTP/magic-link and JWT tokens. The application `users` table below is the **profile/role table**, linked to the Supabase auth user (e.g., by `auth.uid()`), storing `role`, `store_id`, and status for RBAC.

```
users            (id UUID PK = auth.uid(), name, role ENUM, store_id FK?, status, created_at, updated_at)  -- profile/role; auth in Supabase auth.users
stores           (id, name, city, address, status)
warehouses       (id, name, address, status)
locations        (id, type ENUM(store,warehouse), store_id?, warehouse_id?, status)   -- unified view
categories       (id, name, parent_id?, status)
units            (id, name)
products         (id, sku_code UNIQUE, name, description, category_id FK, unit_id FK,
                  cost_price DECIMAL(12,2) MASKED, sale_price DECIMAL(12,2), status, created_at)
suppliers        (id, name, contact_person, phone, email, address, lead_time_days, status)
supplier_products(supplier_id FK, product_id FK, PRIMARY KEY(supplier_id, product_id))
inventory        (product_id FK, location_id FK, qty_on_hand INT, PRIMARY KEY(product_id, location_id))
safety_stock_rules(product_id FK, location_id FK?, safety_stock INT, reorder_point INT,
                   target_level INT, auto_order_enabled BOOL)
purchase_orders  (id, po_number UNIQUE, supplier_id FK, destination_id FK, status ENUM, order_date,
                  expected_date, received_date?, created_by FK, approved_by FK?)
po_lines         (id, po_id FK, product_id FK, qty INT, received_qty INT DEFAULT 0)
sales            (id, store_id FK, sale_datetime, total DECIMAL(12,2), status ENUM(active,voided), created_by FK)
sale_lines       (id, sale_id FK, product_id FK, qty INT, unit_price DECIMAL(12,2), line_total DECIMAL(12,2))
stock_movements  (id, product_id FK, location_id FK, type ENUM(in,out,transfer,adjust,sale_refund),
                  qty INT, ref?, reason?, created_by FK, created_at)   -- audit
alerts           (id, type ENUM, product_id?, location_id?, message, target_role ENUM, read BOOL, created_at)
ai_recommendations(id, product_id FK, location_id FK, type ENUM(warehouse_level, safety_stock),
                   recommended_value INT, reasoning TEXT, accepted STatus, created_at)
audit_logs       (id, actor_id FK, action, entity, entity_id?, detail JSON, created_at)
```

6.2 Indexes (recommended)

- `inventory(product_id, location_id)` PK.
- `sales(store_id, sale_datetime)`, `sales(sale_datetime)` for periodic reports.
- `stock_movements(product_id, location_id, created_at)`.

- `po_lines(po_id)`.
- `alerts(target_role, read, created_at)`.

6.3 Data Integrity

- FK constraints on all references.
- Check constraints: `qty_on_hand ≥ 0`, `qty > 0` for movements, `sale line qty > 0`.
- All mutations run in DB transactions.

6.4 Backup

- Provider-managed backups where available on free tier + scheduled exports (nightly CSV/DB dump to object storage). Phase-2 targets: RPO ≤ 24h, RTO ≤ 4h.

---

7. Data Masking Specifications

7.1 Masked Fields (agreed at SRS)

- `products.cost_price`
- Computed `margin` (displayed where applicable)
- Supplier commercial/finance fields (e.g., credit terms)
- Any field flagged sensitive in config

7.2 Masking Layers

1. **Database layer:** sensitive columns stored masked/encrypted (application-level encryption or tokenization; DBAs/read-only users cannot see raw values).
2. **Application/API layer:**
   - Authorized roles (ADMIN and explicitly authorized) receive clear-text via a **logged** access path.
   - Unauthorized roles receive masked representation `"$**.**"` / `"••••"` and masked value in exports too.
3. **Logs/exceptions:** masked values never logged; error messages never contain raw sensitive values.

7.3 Masking Tests

- Test: forbidden role API call returns masked value; DB read of column returns masked/encrypted; export respects permission; audit log contains no raw values.

---

8. AI Engine Specifications

8.1 Automated Ordering Flow

Trigger (rerun after every stock-affecting event, plus a nightly sweep):

1. For each product where `qty_on_hand ≤ reorder_point` (at a location):
   - If `auto_order_enabled = false` → create LOW_STOCK alert only; stop.
   - If an open PO exists for `(product, location)` → skip (no duplicate) unless override.
   - If no supplier mapped for the product → create alert `NO_SUPPLIER`; no PO.
2. Compute reorder quantity:

```
forecast = demand_forecast(product, horizon = effective_lead_time_days)
open_po = Σ open PO quantities (unreceived)
reorder_qty = max(0, target_level − qty_on_hand − open_po)
if forecast available AND auto_qty_mode='forecast':
    reorder_qty = max(reorder_qty, ceil(forecast))
```

3. Create PO (status DRAFT) to product's mapped supplier, `expected_date = today + lead_time`.
4. If `auto_approve = true` → SENT; else await Admin approval (low-stock alert with pending PO).

Edge cases:

- No sales history for product → fall back to `target_level − on_hand` (no forecast factor).
- Lead time unknown → use default 7 days + alert `MISSING_LEAD_TIME`.
- Multiple products, same supplier → batch into one PO where configured.

8.2 Warehouse Stock-Level Recommendation

For each (product, warehouse):

1. Forecast demand per unit time from historical sales (window default 90 days; min 14 days of history required; else "insufficient data" fallback).
2. `recommended_level = forecast × (lead_time + safety_buffer)` where `safety_buffer` = configurable (default 20%).
3. Clamp to `[safety_stock, target_level]` bounds; if unset, propose `2 × forecast` default.
4. Present with rationale string, e.g. *"Based on last 90 days, expected ~120 units/week. Recommend 240 units to cover 1.5-week lead time + 20% buffer."*
5. User action: ACCEPT (writes to safety_stock_rules / inventory target), MODIFY (edits then accept), REJECT (logged).

8.3 Forecasting Method (v1.0)

- The AI service is a **Node.js module that calls the Groq API** (LLM inference) for demand forecasting and recommendation reasoning.
- Deterministic inputs (recent sales, window = 90 days default, min 14 days required) are passed to Groq; fallback to **simple moving average / exponential smoothing** in the Node backend if no LLM output or insufficient data.
- Trend/seasonality = optional refinement if data volume supports it (scope limited to auto-ordering + warehouse recommendation — BRD L-21).
- All AI inputs, computations (deterministic + Groq output), and outcomes recorded in `ai_recommendations` for audit.
- **Groq availability/resilience:** the AI reasoner is advisory; if the Groq API is unavailable (or free-tier rate limits are hit), the system falls back to deterministic forecasts and still functions (alerts/auto-order still work via rules).

8.4 AI Constraints

- AI is **advisory**; never executes an order without the auto-order consent flag.
- AI never bypasses RBAC or masking.
- Every recommendation explainable and auditable.
- **Groq API key** is stored as a secret environment variable (Render); never logged or committed.
- AI must **degrade gracefully** (deterministic fallback) if the Groq API is unavailable or rate-limited — core inventory/sales/alerts must not depend on the LLM.

---

9. Non-Functional Specifications

ID	Category	Specification (measurable)
NFR-1	Performance	Dashboard/inventory views ≤ 3s p95 under normal load.
NFR-2	Performance	Data reflected in reports ≤ 5 min (near real-time).
NFR-3	Scalability	3 stores, multiple warehouses, ≥ 50 concurrent users.
NFR-4	Availability	Phase-1 free tier: best effort (sleep/cold-start accepted). Phase-2: 99.5%.
NFR-5	Security	TLS 1.2+; encryption at rest.
NFR-6	Security	Authentication & OTP handled by **Supabase Auth**; Node backend validates Supabase JWTs for RBAC; rate-limit/lockout configured in Supabase Auth.
NFR-7	Data Protection	Masking per Section 7.
NFR-8	Usability	Responsive UI; ≤3 clicks to core actions; clear labels.
NFR-9	Maintainability	Modular code, documented, linted; CI on push.
NFR-10	Reliability	No data corruption on partial failure; DB transactions; error envelope.
NFR-11	Auditability	Append-only audit log for significant actions.
NFR-12	Backup	Provider backups (free tier) + scheduled exports; Phase-2 RPO≤24h RTO≤4h.
NFR-13	Compliance	Data-protection best practices on chosen providers.
NFR-14	Accessibility	Clear UI; single language (English).

---

10. Security Requirements

1. HTTPS only; HSTS enabled.
2. Authentication & password/OTP handling via **Supabase Auth** (no plaintext; passwords managed by Supabase).
3. **JWT issued by Supabase Auth**, validated by Node backend (RBAC + role middleware) on every protected route.
4. Rate limiting & lockout on login (configured in **Supabase Auth**); global API rate limit per IP on the Node backend.
5. Input validation on all inputs (server-side), parameterized SQL via the Supabase/Postgres client.
6. Masking of sensitive fields (Section 7).
7. CORS restricted to frontend origin; security headers (CSP, X-Frame-Options).
8. Dependency scanning in CI; `.env` secrets never committed — **Supabase** and **Groq** keys stored as env vars on **Render**.
9. Audit logging of significant actions.
10. No logging of raw sensitive data or credentials.

---

11. UI/UX Requirements (Per Screen)

> Screens are listed with their purpose, key controls, and role visibility. Role symbols: A=Admin, SS=Store Staff, SP=Sales Personnel, SK=Senior Stakeholder.

11.1 Login

- Email + password **or email OTP (magic link)** sign-in (Supabase Auth); show role-aware redirect after login; error on invalid creds; lockout message if locked.

11.2 Dashboard (role-aware)

- A/SK: inventory health KPIs + sales summary + low-stock alerts (widgets per FR-DSH).
- SS: own-store stock status + quick actions (stock-in/out/transfer).
- SP: own sales period + store performance.
- Widgets clickable → drill-down grid.

11.3 Product Master (A)

- Grid: SKU, name, category, unit, sale price, cost price (masked to non-authorized roles), status. Search/filter/pagination. Create/edit modal. Archive button (confirm).

11.4 Inventory (Stock) Screen (A, SS own-store)

- Per-location stock grid: product, on-hand, safety, reorder, target, status badge, last-updated.
- Actions: Stock-In, Stock-Out, Transfer, Adjust (SS limited to own store).
- Each action = form modal: product, qty, (from/to) location, reason/notes; shows live validation and post-mutation feedback.

11.5 Sales Screen (A, SP)

- Record sale: store, date, line-item table (product autocomplete, qty, price), computed totals; validate & save; shows stock sufficiency errors inline.
- Sale history list with detail.

11.6 Sales Reports (A, SP, SK)

- Tabs: Daily / Quarterly / Yearly. Filters: store, product, date range/quarter/year. Table + charts (value & units). Export CSV/PDF.

11.7 Safety Stock / Reorder Config (A)

- Table per product-location: safety, reorder, target, auto-order toggle. Edit inline/modal. “Suggest by AI” button → shows AI recommendation modal (accept/modify/reject).

11.8 Supplier Master (A)

- Grid + create/edit; product mapping; supplier performance score column (value-add).

11.9 Purchase Orders (A; SS = receive)

- PO list: number, supplier, destination, status, expected date, progress. PO detail with line items and receive button (records goods-in). Create manual PO form.

11.10 Alerts Center (A, SS, SP role-scoped)

- Unread/read lists; filter by type; click-through to relevant record.

11.11 Executive Dashboard (SK, A)

- Consolidated: total stock value, turnover, low/out counts, day/qtr/yr sales, store comparison charts; drill-down to store/product.

11.12 Reports Export

- Consistent CSV/PDF export with column set per report; respects masking.

11.13 User Management (A)

- User table, create/edit, role & store assignment, activate/deactivate, password reset.

11.14 Audit Log Viewer (A)

- Filters (actor, action, date, entity); read-only display.

11.15 Onboarding Wizard (A)

- 6-step wizard (BRD §10.17) with progress + skip/resume.

11.16 AI Recommendation Center (A, SK)

- Pending & history of AI recommendations with rationale; accept/modify/reject actions.

---

12. Use Cases

UC-01 Record a Sale (Primary: SP)

Precondition: SP logged in. Main: choose store → add items → save → stock decreases → sales totals update → low-stock evaluation. Post: sale immutable; audit logged. Alternate: insuffficient stock → blocked with product detail.

UC-02 Stock-In / Receive Goods (SS, A)

Both ad-hoc stock-in and PO-linked receipt. Post: stock increases; PO status updates.

UC-03 Auto-Reorder at Reorder Point (System + A)

Trigger when qty ≤ reorder. Auto-PO created per §8.1. Alerts: LOW_STOCK + pending PO to Admin.

UC-04 Generate AI Warehouse Recommendation (A/SK)

Select warehouse/product set → view rationale → accept/modify/reject → stored to audit.

UC-05 Cycle Count Adjustment (SS/A)

Enter counted qty → see variance → confirm → stock updated + audit reason.

UC-06 Transfer Between Locations (SS/A)

Choose from/to → validate → both rows updated atomically → audit.

UC-07 Run Daily/Quarterly/Yearly Report (SP/A/SK)

Select period & filters → view metrics → export.

UC-08 Manage User & Roles (A)

Create/edit user with role+store; deactivate; permission enforcement verified.

UC-09 Onboarding New Product (A)

Create SKU → set safety/reorder/target → map supplier + lead time + auto-order → initial stock → tracked.

UC-10 View Executive Dashboard (SK/A)

Consolidated KPIs → drill to store/product.

---

13. Acceptance & Verification (Testability)

Each BRD acceptance criterion (AC-1…AC-14) is verified by one or more **test cases** here (mapped in Section 14).

Test ID	Test Case	Expected Result
T-AC1	View stock for all products across 3 stores + warehouse	Grid shows correct qty & status, ≤3s
T-AC2	Perform Stock-In/Out/Transfer/Adjust	qty updates correctly; no negative stock; audit logged
T-AC3	Record sale → check stock & reports	Sale total correct; stock decremented; daily/qtr/yr reports reflect sale
T-AC4	Configure safety stock; set stock below reorder	LOW_STOCK alert created; reorder evaluation runs
T-AC5	Enable auto-order; set stock ≤ reorder	Auto-PO generated with correct qty (per §8.1)
T-AC6	Complete PO end-to-end	PO status transitions track; goods-in increases stock
T-AC7	Run AI warehouse recommendation	Rationale shown; accept/modify/reject works & logged
T-AC8	Sales personnel view store performance	Per-store + comparison correct
T-AC9	Senior stakeholder opens executive dashboard	KPIs + drill-down work, near real-time
T-AC10	Login & RBAC	Email/password **and** email OTP (magic link) sign-in work (Supabase Auth); users see only permitted data/menus; unauthorized API → 403
T-AC11	Check masking	Cost price masked to unauthorized roles & in DB/exports
T-AC12	Verify hosting	App live on Vercel + Render (Node backend/AI) + Supabase (postgres/auth/storage) free tier; DB working
T-AC13	Performance	Dashboards ≤3s p95; reports reflect within 5 min
T-AC14	Out-of-scope confirm	Client sign-off in BRD §9

---

14. Traceability Matrix

BRD ID	SRS Section	Test
BR-01 / FR-INV-03	§4.1.3	T-AC1
BR-02 / FR-INV-04–07	§4.1.4–7	T-AC2
BR-03 / FR-SAL-02	§4.2.2	T-AC3
BR-04 / FR-SST	§4.3	T-AC4
BR-05 / FR-AI-01	§8.1	T-AC5
BR-06 / FR-AI-02	§8.2	T-AC7
BR-07 / FR-INV-04…07	§4.1.4–7	T-AC2
BR-08 / FR-SAL-03	§4.2.3	T-AC8
BR-09 / FR-MON-02	§11.11	T-AC9
BR-10 / Data	§6	T-AC12
BR-11 / FR-SEC	§7	T-AC11
BR-12 / Hosting	§2.3, §5	T-AC12
BR-13 / 3 stores	§4.1.2	T-AC1
BR-14 / FR-USER	§4.6	T-AC10
BR-15 / Alerts	§4.1.8	T-AC4
BR-16 / Audit	§4.5.8	T-AC2
Value-add modules	§4.5	Partial suite

15. Open / TBD Items

#	Item	Decision Needed By	Owner
~~TBD-1~~	~~Backend: Node.js vs Python~~ — **DECIDED: Node.js (BRD v3.4)**	Resolved	SA (Anoop) + Tech Lead
~~TBD-2~~	~~Component & charting libraries~~ — **DECIDED: shadcn/ui + Recharts (BRD v3.4)**	Resolved	SA/Tech Lead
TBD-3	Exact masked-field list confirmed by client	SRS sign-off	Client
TBD-4	AI auto-qty mode default (target-based vs forecast-based)	Sprint 3	SA
TBD-5	Perishable (FR-EXP) inclusion in v1.0 (Could)	Sprint planning	SM/PM
TBD-6	Alert email provider/limit on free tier	Sprint 3	Tech Lead (may use Supabase Auth email / provider SMTP)

Approval Sign-off

Role	Name	Signature	Date
Client / Sponsor	Prof		
Project Manager	Laxman Patel		
Business Analyst	Ved Naik		
Solutions Architect	Anoop Gupta		
Scrum Master	Devdarshan S		
Tech Lead	Rohan Vashisht		

End of SRS — Version 1.0 · Project: Vaultory · Team: Vaultory